P3146R1
Clarifying std::variant converting construction

Published Proposal, 2024-02-20

Author:
Giuseppe D'Angelo

Audience:
LEWG, LWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper aims at clarifying the behavior of `std::variant`'s converting constructor and converting assignment operator in some corner cases. These cases have been made possible by the adoption of [P2280R4] ("Using unknown pointers and references in constant expressions"). The clarification has an impact on the design of [P0870R5].

Table of Contents

§ 1. Changelog

- R1
  - Amend the discussion regarding the C++17-compatible implementation strategy.
  - Minor fixes.
- R0
  - First submission.

§ 2. Motivation and Scope

Consider this example:

```
using IC = std::integral_constant<int, 42>;

IC ic;
std::variant<float> v = ic;
```

All major implementations reject this code (Godbolt). However it is not entirely clear if the code shouldn't instead be well-formed, and the variant containing `42` converted to `float`.

✧ ✧ ✧

The current wording for `std::variant`'s converting constructor (and similarly for its converting assignment operator) has been established by [P0608R3], "A sane variant converting constructor". In the latest Standard draft, [variant.ctor/14] states:

> `template<class T> constexpr variant(T&& t) noexcept(see below);`
>
> Let `Tj` be a type that is determined as follows: build an imaginary function `FUN(Ti)` for each alternative type `Ti` for which `Ti x[] = {std::forward<T>(t)};` is well-formed for some invented variable `x`. The overload `FUN(Tj)` selected by overload resolution for the expression `FUN(std::forward<T>(t))` defines the alternative `Tj` which is the type of the contained value after construction.

Let's therefore try to build the `FUN` overload set for the `std::variant<float>` example above. Checking whether the `x` variable is well-formed can be implemented using the following concept:

```
template <typename Ti, typename From>
concept FUN_constraint = requires(From &&from) {
    { std::type_identity_t<Ti[]>{ std::forward<From>(from) } };
};
```

There is only one alternative type (`float`), so the associated `FUN` overload looks like this:

```
template <typename T>
    requires FUN_constraint<float, T>
void FUN(float);
```

Now, given the construction above:

```
std::variant<float> v = ic;
```

we therefore need to check if this is well-formed:

```
// In variant::variant(T &&t) constructor; therefore:
//   T = IC &
//   t is lvalue reference to IC
```

```
FUN<T>(std::forward<T>(t)); // well-formed?

// or, equivalently:
FUN<IC &>(ic);              // well-formed?
```

The answer before the adoption of [P2280R4] ("Using unknown pointers and references in constant expressions") was **no**: the call would not find the `FUN(float)` overload because its associated `FUN_constraint<float, IC &>` constraint would not be satisfied. Since there is no viable `FUN` overload, it follows that `std::variant<float> v = ic` is ill-formed.

The purpose of the check in `std::variant`'s converting constructor is to exclude narrowing conversions (cf. [P0870R5]). To this end, the check done with the invented `x` variable uses a form of list-initialization specifically because list-initialization bans narrowing conversions.

The wording in [variant.ctor/14] makes `x` an array, so that its list-initialization performs aggregate initialization, and [dcl.init.aggr]/4 applies:

> If that initializer is of the form *assignment-expression* or = *assignment-expression* and a narrowing conversion ([dcl.init.list]) is required to convert the expression, the program is ill-formed.

Going back to the opening example: initializing the `float` object in the `x` array from a value of type `IC` requires a conversion sequence. First the value is converted to `int` through `IC`'s conversion operator; then the `int` value so obtained is converted to `float` via a floating-integral conversion ([conv.fpint]/2). Such a conversion is a narrowing conversion ([dcl.init.list]/7.3), "except where the source is a constant expression and the actual value after conversion will fit into the target type and will produce the original value when converted back to the original type".

Is the source a constant expression in this case? Before [P2280R4], the answer was always no, because of the usage of a *reference type* in the concept's *requires-expression* 's argument list (the type of `from`).

That is, even if:

- `IC`'s conversion operator towards `int` returns a compile-time constant value (the value of its `value` static data member, which in turn comes from its template parameter), and never actually reads the value of `t` (`from`) through the reference in order to perform the conversion; and
- `IC`'s conversion operator is also `constexpr`, so it can be used during constant evaluation; and

- `42` is convertible to `float` and back to `int` without any loss of information,

the pre-[P2280R4] rules in [expr.const] made it a non-constant expression to merely "mention" `from`. This triggers a narrowing conversion (as we no longer are in the "except where" case), making the initialization of the array of `float`s ill-formed, and therefore the `FUN_constraint<float, IC&>` constraint not satisfied.

With the adoption of [P2280R4] some of these limitations have been lifted. GCC 14, which implements [P2280R4], accepts the `FUN<IC &>(ic);` call (Godbolt). At the time of this writing, GCC 14 is also the *only* compiler implementing the new rules.

§ 2.1. Why doesn't the testcase work?

Given this premise, then come that `std::variant<float> v = ic;` is still ill-formed in GCC 14? The answer lies in the implementation of `std::variant`.

`std::variant` is C++17, and therefore predates concepts. All major Standard Library implementations use SFINAE to constrain the set of `FUN` functions and find the right alternative type to build. In their "SFINAE triggers", they employ a call to `std::declval<T>()` as the single element in the array of `Ti`s, something along these lines:

```
// Scheme of SFINAE-based narrowing detection, cf. P0870
template <typename T>
struct NarrowingDetector { T x[1]; };

template <typename From, typename To, typename = void>
constexpr inline bool is_convertible_without_narrowing_v = false;

template <typename From, typename To>
constexpr inline bool is_convertible_without_narrowing_v<From, To,
    std::void_t<decltype(NarrowingDetector<To>{ { std::declval<From>() } }
                                                // ^^^^^^^^^^^^^^^^^^^^
    = true;

// Example usage for constraining FUN(float):
template <typename From,
          std::enable_if_t<is_convertible_without_narrowing_v<From, float>,
    void FUN(float);
```

Since `std::declval` itself is not a constexpr function, calling it and using the return value is not a constant expression, and that ends up triggering a narrowing conversion; `FUN<IC &>(ic)` becomes unviable (and, consequently, `std::variant<float> v = ic;` becomes ill-formed.)

But the Standard never talks about using `std::declval` for doing this detection! As shown above, doing the same detection using concepts will allow for the construction to succeed.

This line of reasoning has resulted in this bug report against libstdc++, and then, ultimately, in this paper, in order to clarify the behavior of `std::variant`'s converting constructor.

§ 2.2. Proposed change

In conclusion: there is a gap between the specification of `std::variant`'s converting constructor, and what implementations actually do.

We see two possibilities: to amend `std::variant`'s specification so that

1. either the introductory example (`std::variant<float> v = ic`) is *supposed to work*, and therefore current implementations must be fixed in other to support it; or
2. to endorse the fact that the construction of `x` in `Ti x[] = {std::forward<T>(t)};` (as described by [variant.ctor/14]) is supposed to happen *outside* of a constant evaluation context, and the current behavior by implementations is the expected one.

This paper proposes the first option.

The rationale is that the wording for narrowing conversions in the core language is specially crafted to take constant expressions into account; and the wording for `std::variant` converting constructor wants to avoid narrowing conversions. In case one can determine that a conversion can happen without narrowing, then there is no reason for `std::variant` to reject it. In other words, **the behavior of `std::variant` should not diverge from the core language specification of what does what doesn't constitute a narrowing conversion.**

The second option would also go against the possible future evolution of having `constexpr` function arguments (cf. [P1045R1]), and for `std::variant` to truly behave like builtin types w.r.t. narrowing conversions:

```
constexpr int i = 42;
float f{i};                 // OK
```

```
std::variant<float> v{i};  // Ill-formed. No change proposed (or possible,
```

§ 3. Design decisions

§ 3.1. If we allow the example to work, would some user code break?

The behavior made possible by [P2280R4] differs from the one currently implemented in the sense that the set of viable `FUN` overloads (for a given variant specialization and input type) is going to be *the same, or bigger*: certain conversions are no longer considered narrowing, and therefore include the respective `FUN` overloads in the overload set used to determine which alternative type is the active one.

In other words: it will never be the case that a `FUN` overload that is viable according to the current implementations will no longer be; [P2280R4] strictly *relaxes* the constraints on `FUN`.

There is a risk associated with extending the set of viable `FUN` overloads: if more than one overload becomes viable, then overload resolution has to pick the best one. This could break some user code, or change its behavior.

§ 3.1.1. Variant construction was ill-formed; becomes legal

This is the very first example of this paper:

```
using IC = std::integral_constant<int, 42>;

IC ic;
std::variant<float> v = ic; // now     : ill-formed
                            // proposed: well-formed
```

Of course one can concoct examples where code misbehaves in case `std::is_convertible_v<IC, std::variant<float>>` becomes `true`, but there is no real-world scenario where this would actually be harmful.

("Clever" user code is already able to detect pretty much any interface change in the Standard Library, via concepts and/or SFINAE. That does not imply that the Standard Library isn't allowed to evolve, see also [SD-8].)

Note that some ill-formed code may stay ill-formed, although for a different reason:

```
std::variant<float, double> v = ic;  // now     : ill-formed (no viable FUN
                                     // proposed: ill-formed (ambiguous)
```

3.1.2. Variant construction was legal; becomes ill-formed

In this case, it means that there was one single best `FUN` overload; by relaxing the rules, we end up with multiple overloads that rank equal, so the call to `FUN` becomes ambiguous and `std::variant` construction becomes ill-formed.

For instance:

```
IC ic;
std::variant<long, float> v = ic;  // now     : selects long
                                   // proposed: ill-formed (ambiguous)
```

We actually **welcome** this change, because it is highlighting a semantic bug in user code. According to core language there is no reason why `long` should be preferred here: the two conversions from `ic` to `long` and to `float` are equally ranked, and neither is narrowing:

```
void f(long);
void f(float);

f(ic); // ERROR, ambiguous
```

3.1.3. Variant construction was legal, stays legal, but changes semantics

The most "dangerous" case for end-users would be the case where the alternative type selected by `std::variant`'s converting constructor silently changes.

That is:

```
From f;
std::variant<A, B> v = f;  // now  : selects A
```

```
                                    // after: selects B. Is this possible?
```

We do not believe that this is a possibility.

> NOTE: for the sake of the argument, we are not taking into consideration the case where `A`/`B`/`From`'s own behavior is influenced by [P2280R4], for instance by having a `B::B(From)` converting constructor with a constrain that itself changed meaning due to [P2280R4].

We are going once more to refer to:

- `FUN` as the imaginary functions described by `std::variant`'s converting constructor;
- its associated constraint `FUN_constraint`, where one checks if `Ti x[] = {std::forward<T>(t)};` is well-formed. `Ti` is each alternative type in the variant, `T` and `t` are the parameters of `std::variant`'s converting constructor (i.e. `From &` and `f` in the example).

Our reasoning is as follows:

- For the above code to select `B`, then `FUN(A)` and `FUN(B)` are viable, meaning `From` converts to both `A` and `B` and the `FUN_constraint` is satisfied for both. Note that `FUN(A)` was always viable, as `A` was selected before. This means that either `FUN(B)` was not viable before [P2280R4] and/or it changed its ranking in overload resolution.
- [P2280R4] does not affect overload resolution. Therefore, `FUN(B)` was not viable before, and now it is, and ranks better than `FUN(A)`.
- [P2280R4]'s impact in `FUN_constraint` is limited to allowing certain conversions that were previously classified as narrowing (and thus ill-formed, making the constraint unsatisfied). It does not affect any other kind of conversion.
- Therefore, `FUN(B)` was not viable because there was a narrowing conversion between `From` and `B`.
- Narrowing can only happen between certain scalar types (cf. [dcl.init.list/7]). In a user-defined conversion sequence, narrowing can only happen during a standard conversion sequence.
- Therefore, at least one between `From` and `B` must be of scalar type.
    - If they're both class types, then there's a user-defined conversion from `From` to `B`, which is never narrowing.
    - There cannot be a conversion sequence like `From` → scalar type → standard conversion to another scalar type → `B` as that would require two user-defined conversions (first and last conversions).

- `From` must be of class type, otherwise [P2280R4] has no effect on narrowing.
    - If `From` is a scalar type, then the `FUN_constraint` check won't change meaning after [P2280R4]. If the conversion to `B` was narrowing before, it is still narrowing afterwards, as we cannot apply the "source is a constant expression" exception to the narrowing rule, because that would require reading the value of the `From` source *through the reference* (`t`), and that is still not allowed in a constant expression even after [P2280R4].
- It follows that `B` must be of scalar type (otherwise, again, there would be no narrowing happening). There's a user-defined conversion sequence between `From` and `B`.
- The user-defined conversion sequence (UCS) from `From` to `B` must include a standard conversion sequence (SCS) where narrowing was happening before [P2280R4].
- Narrowing can only happen in a SCS of Conversion rank (cf. [over.ics.scs]), which is the lowest rank possible.
    - A UCS is composed of a SCS, a user-defined conversion, and a second SCS.
    - The first SCS must be of rank Exact Match, as no other rank applies to classes.
    - Therefore, the narrowing was happening in the second SCS, which is the one of Conversion rank.
- The conversion from `From` to `A` must be ranked *worse* than the conversion from `From` to `B`.

We have reached the contradiction: **this is impossible**, because there is simply no way for the conversion from `From` to `A` to be worse. At most, it can be of equal rank.

- To show this, given the UCS from `From` to `B`, then the conversion from `From` to `A` must also be a UCS, whose second SCS is also of Conversion rank. (The first SCS is again Exact Match, as no other applies to a class type, and user-defined conversions don't have a rank.)
- Since the ranking is equal, then [over.ics.rank/4] applies, which says that "Two conversion sequences with the same rank are indistinguishable unless one of the following rules applies". All the rules listed exclude the possibility that a narrowing for the conversion to `B` was happening before [P2280R4], and also that no narrowing was happening for `A`:
    - 4.1: "A conversion that does not convert a pointer or a pointer to member to `bool` is better than one that does": this would imply `A` is `bool`, meaning that narrowing conversion was already happening for `F(A)` (a pointer to bool conversion is always narrowing), which is impossible, as it satisfies `FUN_constraint` by hypothesis.
    - 4.2: "A conversion that promotes an enumeration whose underlying type is fixed to its underlying type is better than one that promotes to the promoted underlying type, if the two are different": in neither case there is narrowing.

- 4.3: "A conversion in either direction between floating-point type `FP1` and floating-point type `FP2` is better than a conversion in the same direction between `FP1` and arithmetic type `T3` if the floating-point conversion rank ([conv.rank]) of `FP1` is equal to the rank of `FP2` [...]", which excludes the possibility of that there was a narrowing conversion between `From` and `B` (there can't be if they're both floating point types with equal rank).
- 4.4 and subsequent only apply between pointers conversions, where there is no narrowing as per language rules.

In all the last cases we have reached a contradiction: the reasoning starts with `FUN(A)` having its constraint satisfied (no narrowing), and `FUN(B)` having its one unsatisfied (narrowing); and the conclusions contradict this starting point.

Therefore, this breaking change cannot happen.

§ 3.1.4. Conclusions

In conclusion, the change proposed by this paper may result in source code becoming ill-formed in cases where there was an ambiguity to begin with. In any other case there is no behavioral difference.

Once more, due to the lack of `constexpr` function parameters (cf. [P1045R1]), the following code won't change meaning:

```
constexpr int i = 42;
float f{i};                 // OK, no narrowing
std::variant<float> v{i};   // Still ill-formed
```

§ 3.2. What about compilers implementing [P2280R4] in C++17?

[P2280R4] has been proposed as a Defect Report, all the way back against C++11. When using `std::variant` in C++17 mode, an implementation that implements [P2280R4] will not be able to use concepts (like the `FUN_constraint` shown above) in order to express the constraints on its overloads of `FUN`; it must fall back to SFINAE or similar detections.

A viable C++17 implementation strategy has been suggested by Jiang An here (many thanks!): one could hide the fact that `std::declval` isn't usable in a constant expression behind one layer of indirection.

The detection previously shown could be amended like this (Godbolt):

```
template <typename T>
struct NarrowingDetector { T x[1]; };


// Indirection layer to hide std::declval:
template <typename To, typename From>
auto is_convertible_without_narrowing_helper(From &&f) ->
    decltype(NarrowingDetector<To>{ {std::forward<From>(f)} });


// As before:
template <typename From, typename To, typename = void>
constexpr inline bool is_convertible_without_narrowing_v = false;

template <typename From, typename To>
constexpr inline bool is_convertible_without_narrowing_v<From, To,
    std::void_t<decltype(is_convertible_without_narrowing_helper<To>(std::
    = true;


// Example usage for constraining FUN(float):
template <typename From,
          std::enable_if_t<is_convertible_without_narrowing_v<From, float>,
    void FUN(float);


// Testcase:
int main()
{
    using IC = std::integral_constant<int, 42>;

    FUN<IC>(IC{}); // OK after P2280R4

    IC ic;
    FUN<IC &>(ic); // OK after P2280R4
}
```

§ 4. Impact on the Standard

This proposal clarifies the behavior of `std::variant`'s converting constructor and converting assignment operator.

It proposes no other changes to the Standard Library or to the core language.

§ 5. Technical Specifications

§ 5.1. Proposed wording

We are not proposing any normative changes: the current wording is already correct.

At LEWG/LWG's discretion, a non-normative note could be added to [variant.ctor/14] and/or to [variant.assign/11] to state that initialization of the invented variable `x` happens in a core constant expression if possible; or, similarly, the first example could be added, explaining the expected behavior. The following text provides the wording for such an example.

✧ ✧ ✧

In [variant.ctor], add the following example to the specification for the

```
template<class T> constexpr variant(T&& t) noexcept(see below);
```

constructor:

> [*Example 1*:
>
> ```
> variant<string, bool>  v1 = "meow"; // holds string
> variant<float, long>   v2 = 0;      // holds long
>
> using IC = integral_constant<int, 42>;
> variant<float, string> v3 = IC{};   // holds float
> ```
>
> — *end example*]

§ 6. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[GCC-BR-113060]

Bug 113060 - std::variant converting constructor/assignment is non-conforming after P2280?. URL: https://gcc.gnu.org/bugzilla/show_bug.cgi?id=113060

[P0608R3]

Zhihao Yuan. *A sane variant converting constructor*. 3 October 2018. URL: https://wg21.link/p0608r3

[P0870R5]

Giuseppe D'Angelo. *A proposal for a type trait to detect narrowing conversions*. 15 February 2023. URL: https://wg21.link/p0870r5

[P1045R1]

David Stone. *constexpr Function Parameters*. 27 September 2019. URL: https://wg21.link/p1045r1

[P2280R4]

Barry Revzin. *Using unknown references in constant expressions*. 11 April 2022. URL: https://wg21.link/p2280r4

[SD-8]

Bryce Lelbach. *SD-8: Standard Library Compatibility*. URL: https://isocpp.org/std/standing-documents/sd-8-standard-library-compatibility